

UNIT I – INTRODUCTION TO JAVA PROGRAMMING & FUNDAMENTALS

Overview of Java platform and features: JVM, JRE, JDK , Java program structure, compiling and running Java programs , Data types, variables, operators, type casting , Control structures: if-else, switch, loops (for, while, do-while) , Arrays – single and multidimensional , String handling and built-in functions (String, StringBuffer, StringBuilder) , Introduction to Exception Handling (try, catch, finally, throw, throws).

Applications: Basic calculators, string processors, and array-based apps.

1.1 Overview of Java platform and features:

Java is a **programming language** and a **platform**. Java is a high level, robust, object-oriented and secure programming language.

Java was developed by *Sun Microsystems* (which is now the subsidiary of Oracle) in the year 1995.

James Gosling is known as the father of Java.

Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java. Any hardware or software environment in which a program runs is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

1. Object-oriented:

Java is an object-oriented programming language.

Everything in Java is an object.

Object-oriented means we

organize our software as a

combination of different types of

objects that incorporates both

data and behavior.

Object-oriented programming (OOPs) is a methodology that simplifies software development



and maintenance by providing some rules.

2. Simple:

Java is very easy to learn, and its syntax is simple, clean and easy to understand.

Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.

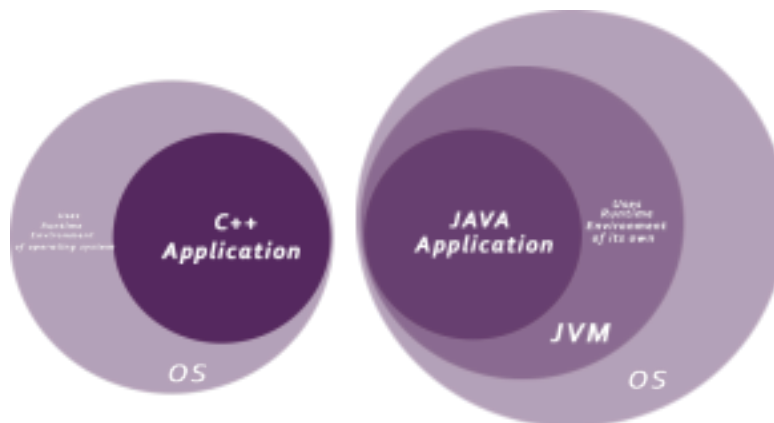
There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

3. Secure:

Java program cannot harm another system thus making it safe. Java is secured because:

- No explicit pointer.

- Java Programs run inside virtual machine sandbox.



4. Platform Independent:

When we write Java code, it is

first compiled by the compiler

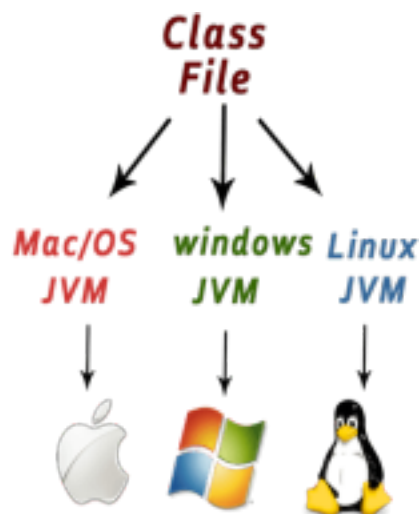
and then converted into

bytecode (which is platform-independent).

This byte code can run on any

platform which has JVM

installed.



5. Robust:

Java is robust because:

It uses strong memory management.

There is a lack of pointers that avoids security problems.

There is automatic garbage collection in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.

There are exception handling and the type checking mechanism in Java. All these points make Java robust.

6. Portable:

Java programs can execute in any environment for which there is a Java run-time system (JVM).

Java programs OR codes can be run on any platform means OS (Linux, Window, Mac). Java programs can be shifted over World Wide Web (e.g., applets)

We may carry the java byte code to any platform.

7. Architecture Neutral:

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

8. Dynamic:

Java is a dynamic language.

It supports dynamic loading of classes. It means classes are loaded on demand.

It also supports functions from its native languages, i.e., C and C++. Java supports dynamic compilation and automatic memory management (garbage collection).

9. Interpreted:

Java code is not directly executed by the computer.

It is first compiled into bytecode.

This byte code is then understood by the JVM. This enables Java to run on any platform without rewriting code.

10. High Performance:

Java is faster than old interpreted languages.

Java program is first converted into bytecode which is faster than interpreted code.

It is slower than fully compiled languages like C or C++ because of interpretation and JIT compilation process.

Java performance is improve with the help of Just-In-Time (JIT) compilation, which makes it faster than many interpreted languages but not as fast as fully compiled languages.

11. Multi-Threaded:

Multithreading in Java allows multiple threads to run at the same time. Java provides build in support for managing multiple threads. A **thread** is known as the smallest unit of execution within a process.

It improves CPU utilization and enhancing performance in applications that require concurrent task execution.

It is especially important for interactive and high-performance applications, such as games and real-time systems.

12. Distributed:

Java is distributed because it facilitates users to create distributed applications in Java.

RMI and EJB are used for creating distributed applications.

This feature of Java makes us able to access files by calling the methods from any machine on the internet.

JDK - Java Development Kit:

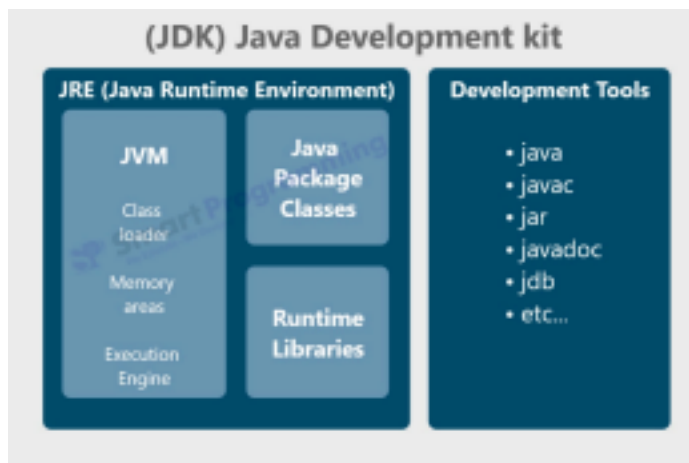
It is a software development environment which is used to develop Java applications.

It physically exists and contains JRE along with development tools. JDK is an implementation of any one of the below java platforms released by Oracle Corporation, which are as follows.

- ❖ Standard Edition Java Platform
- ❖ Enterprise Edition Java Platform
- ❖ Micro Edition Java Platform

In general, JDK contains a private

Java Virtual Machine (JVM) and a few other resources like interpreter/loader, java compiler, archivers (jar files) and Javadoc for the documentation generator to complete the development of Java Applications.



JRE - Java Runtime Environment:

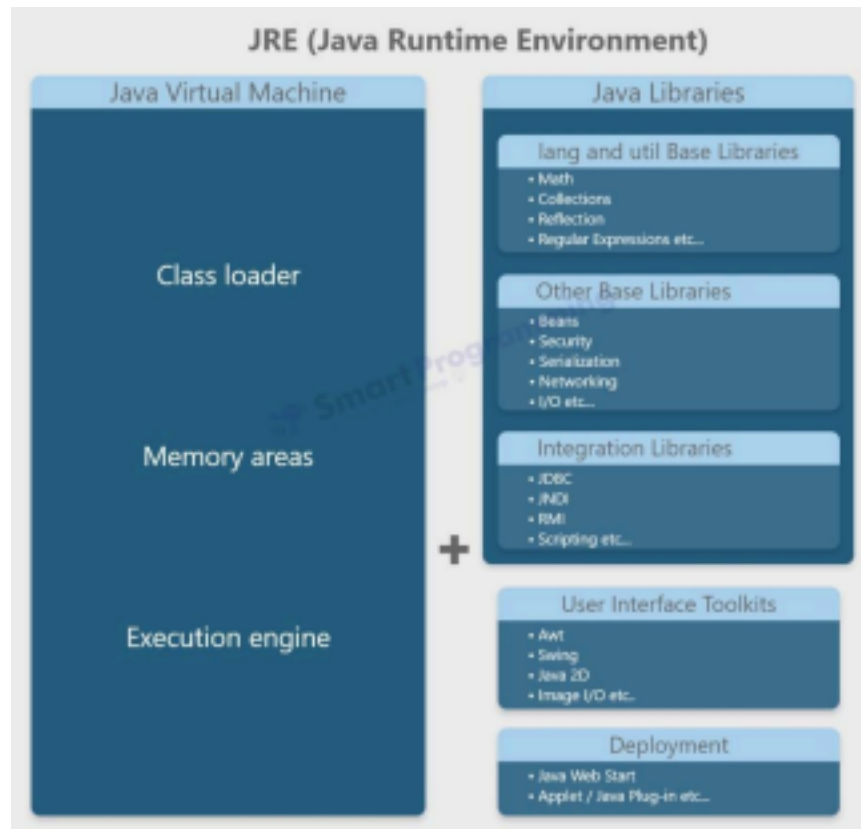
The JRE is a software layer that runs on top of a computer's operating system software and provides the class libraries and other required resources to run a specific java program.

JRE is one of the three interrelated components for developing Java programs along with JDK and JVM.

The JDK and JRE are the two components that interact with each other to create a sustainable run-time environment that enables the seamless execution of Java-based applications in virtually any operating system.

JRE run-time architecture generally uses the ClassLoader, Bytecode verifier, and Interpreter.

In which ClassLoader dynamically load all the necessary classes to run a Java program. It also automates the loading of classes as per requirement, so it can also effectively utilize the memory.



Bytecode verifier checks the format and accuracy of Java code before it passes to the interpreter.

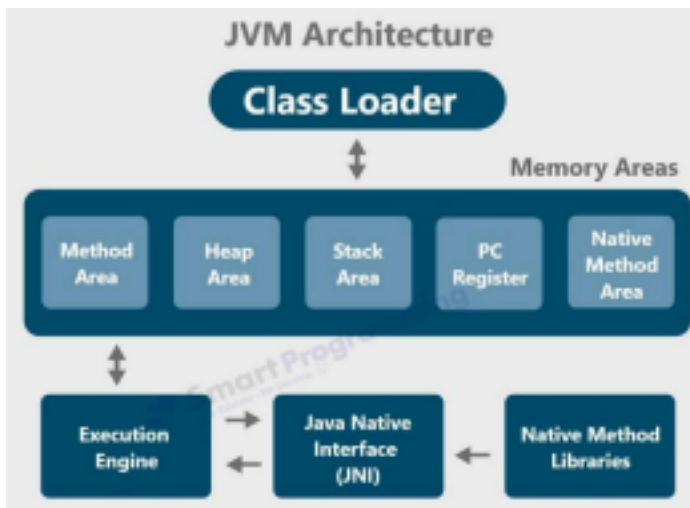
And finally, the interpreter helps to run the java program natively on the underlying machine.

JIT – Just In Time & JVM - Java Virtual Machine:

There are 3 main components in JVM :-

1. **Class Loader:** The class loader is responsible for dynamically organizing classes into namespaces and ensuring they are available for execution.

loading Java classes into memory at runtime. It handles loading, linking, and initialization,



2. **Memory Areas:** The JVM's memory areas include the heap (for objects), stack (for method calls and local variables), method area (for class-level data), and native method stacks. These regions manage memory allocation, storage of program data, and execution context.

3. **Execution Engine:** The execution engine interprets or compiles byte code into machine code for execution by the CPU. It includes the interpreter for line-by-line execution and the Just-In-Time (JIT) compiler for optimizing frequently executed code paths.

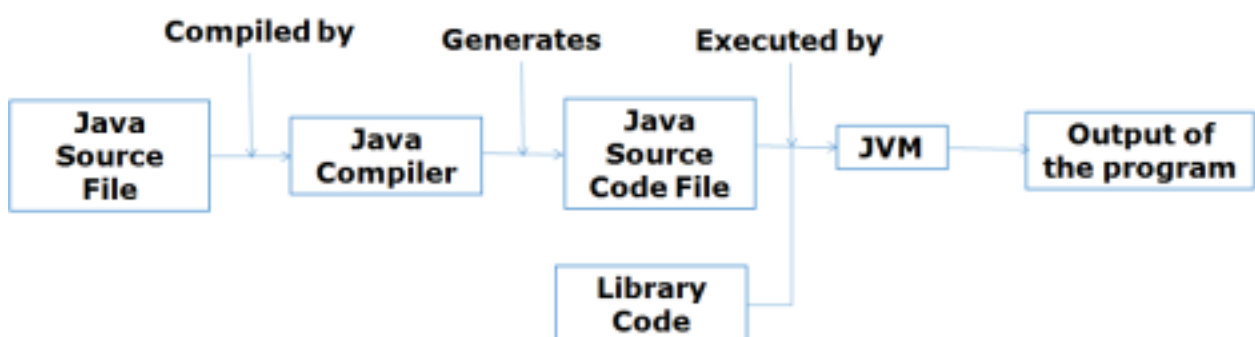
JIT is an integral part of JVM and Garbage Collector.

It is capable of compiling the Java code into machine code, which can significantly improve the performance of Java Applications.

While the compilation of the codes, it is not guaranteed which code will be compiled. JIT usually compile the code as per its requirement to JVM.

Once method execution crosses the certain limit (of approx. 10K execution), then it will be converted into machine code.

To run a simple program in Java programming, we have to use the Java compiler to convert the source code into Java class code. After that, JVM is responsible for running the program.



1.2.STRUCTURE OF JAVA PROGRAM

```
class Simple
{
    public static void main(String args[])
    {
        System.out.println("Java World");
    }
}
```

To compile:

```
javac Simple.java
```

To execute:

```
java Simple
```

class keyword is used to declare a class in java.

public keyword is an access modifier which represents visibility, it means it is visible to all.

static is a keyword, if we declare any method as static, it is known as static method. The core advantage of static method is that there is no need to create object to invoke the static method. The main method is executed by the JVM, so it doesn't require to create object to invoke the main method. So it saves memory.

void is the return type of the method, it means it doesn't return any value. **main** represents the starting point of the program.

String[] args is used for command line argument.

System.out.println() is used print statement.

A program is written in JAVA, the **javac** compiles it. The result of the JAVA compiler is the **.class** file or the **bytecode** and not the machine native code (unlike C compiler). The bytecode generated is a non-executable code and needs an interpreter to execute on a machine. This interpreter is the JVM and thus the Bytecode is executed by the JVM. And finally program runs to give the desired output.

1.3. DATA TYPES

There are two data types available in Java.

Primitive Data Types

There are eight primitive data types supported by Java.

Primitive data types are predefined by the language and named by a keyword.

Let us now look into the eight primitive data types in detail.

byte

Byte data type is an 8-bit signed two's complement integer.

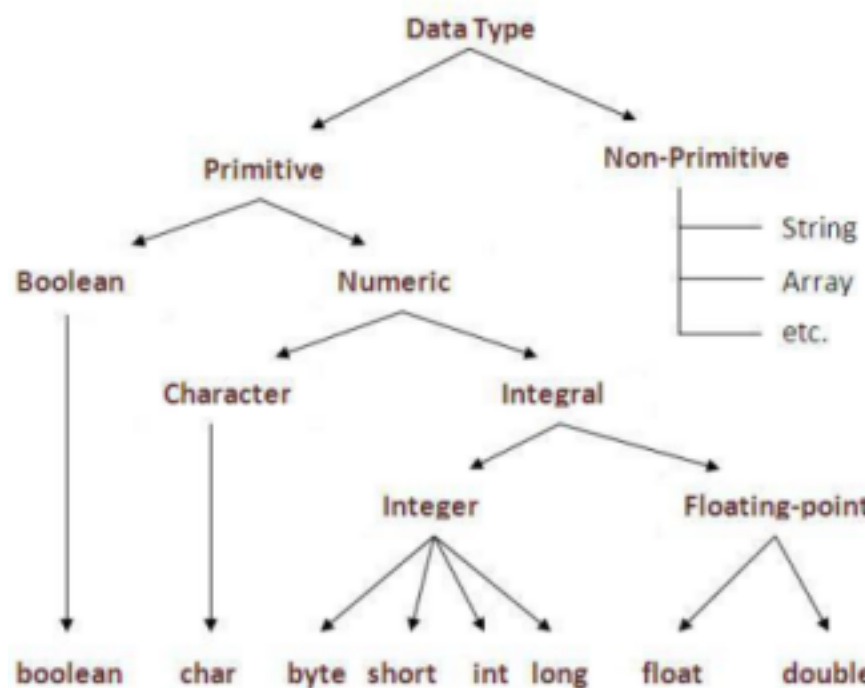
Minimum value is -128 (-2^7).

Maximum value is 127 (inclusive) (2^7-1).

Default value is 0.

Byte data type is used to save space in large arrays, mainly in place of integers, since a byte is four times smaller than an integer.

Example: byte a = 100, byte b = -50.



short

Short data type is a 16-bit signed two's complement integer. Minimum value is -32,768 (-2^{15}).

Maximum value is 32,767 (inclusive) ($2^{15}-1$).

Short data type can also be used to save memory as byte data type. A short is 2 times smaller than an integer.

Default value is 0. -20000.

Example: short s = 10000, short r =
int

Int data type is a 32-bit signed two's complement integer.

Minimum value is - 2,147,483,648 (-2^{31}).

Maximum value is 2,147,483,647(inclusive) ($2^{31} - 1$).

Integer is generally used as the default data type for integral values unless there is a concern about memory.

The default value is 0.

Example: int a = 100000, int b = -200000.

long

Long data type is a 64-bit signed two's complement integer.

Minimum value is -9,223,372,036,854,775,808(-2^{63}).

Maximum value is 9,223,372,036,854,775,807 (inclusive)($2^{63} - 1$).

This type is used when a wider range than int is needed.

Default value is 0L.

Example: long a = 100000L, long b = -200000L.

float

Float data type is a single-precision 32-bit IEEE 754 floating point. Float is mainly used to save memory in large arrays of floating point numbers.

Default value is 0.0f.

Float data type is never used for precise values such as currency. Example: float f1 = 234.5f.

double

double data type is a double-precision 64-bit IEEE 754 floating point. This data type is generally used as the default data type for decimal values, generally the default choice.

Double data type should never be used for precise values such as currency. Default value is 0.0d.

Example: double d1 = 123.4.

boolean

boolean data type represents one bit of information.

There are only two possible values: true and false.

This data type is used for simple flags that track true/false conditions. Default value is false.

Example: boolean one = true.

char

char data type is a single 16-bit Unicode character.

Minimum value is '\u0000' (or 0).

Maximum value is '\uffff' (or 65,535 inclusive).

Char data type is used to store any character.

Example: char letterA = 'A'.

Non-primitive Data Types

In java, non-primitive data types are the reference data types or user-created data types.

All non-primitive data types are implemented using object concepts. Every variable of the non-primitive data type is an object.

The non-primitive data types may use additional methods to perform certain operations.

The default value of non-primitive data type variable is null.

In java, examples of non-primitive data types are String, Array, List, Queue, Stack, Class, Interface, etc.

Variables

A variable is the holder that can hold the value while the java program is executed.

A variable is assigned with a datatype. It is name of *reserved area allocated in memory*.

In other words, it is a *name of memory location*. There are three types of variables in java: local, instance and static.

Declaring a Variable:

Before using any variable, it must be declared.

```
datatype variable_name = value;
```

To declare more than one variable of the specified type, use a comma-separated list.

Example

```
int a, b, c; // Declaration of variables a, b, and c.
```

```
int a = 20, b = 30; // initialization
```

```
byte B = 22; // Declaratrion initializes a byte type variable B. Types of
```

Variable:

There are three types of variables in java:

- ✓ local variable
- ✓ instance variable
- ✓ static variable

Local Variable

Local variables are declared inside the methods, constructors, or blocks. Local variables are created when the method, constructor or block is entered Local variable will be destroyed once it exits the method, constructor, or block. Local variables are visible only within the declared method, constructor, or block.

Local variables are implemented at stack level internally.

There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use. Access specifiers cannot be used for local variables.

Instance Variable

Instance variables are declared in a class, but outside a method, constructor or any block.

Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.

Instance variables hold values that must be referenced by more than one method, constructor or block, or essential parts of an object's state that must be present throughout the class.

Instance variables can be declared in class level before or after use.

Access modifiers can be given for instance variables.

The instance variables are visible for all methods, constructors and block in the class. It is recommended to make these variables as private. However, visibility for subclasses can be given for these variables with the use of access modifiers.

Instance variables have default values.

- o numbers, the default value is 0,
- o Booleans it is false,

- o Object references it is null.

Values can be assigned during the declaration or within the constructor.

Instance variables cannot be declared as static.

Instance variables can be accessed directly by calling the variable name inside the class. However, within static methods (when instance variables are given accessibility), they should be called using the fully qualified name. `ObjectReference.VariableName`.

Static variable

Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block. Static variables are rarely used other than being declared as constants. Constants are variables that are declared as public/private, final, and static. Constant variables never change from their initial value.

Static variables are stored in the static memory. It is rare to use static variables other than declared final and used as either public or private constants.

Static variables are created when the program starts and destroyed when the program stops.

Visibility is same as instance variables. However, most static variables are declared public since they must be available for users of the class.

Default values are same as instance variables.

- o numbers, the default value is 0;
- o Booleans, it is false;
- o Object references, it is null.

1.4 OPERATORS:

Operator in java is a symbol that is used to perform operations. Java provides a rich set of operators to manipulate variables. For example: +, -, *, / etc. All the Java operators can be divided into the following groups –

- ✓ Arithmetic Operators :
- ✓ Relational Operators
- ✓ Bitwise Operators
- ✓ Logical Operators
- ✓ Assignment Operators: =

✓ Ternary operator: ? :

✓ Unary operator

The Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations in the same way as they are used in algebra.

Example:

int A=10,B=20;

Operator	Description	Example	
+ (Addition)	Adds values A & B.	A + B	30
- (Subtraction)	Subtracts B from A	A - B	-10

* (Multiplication)	Multiplies values A & B	A * B	200
/ (Division)	Divides B by A	B / A	2
% (Modulus)	Returns remainder.	B % A	0

// Java program to illustrate arithmetic operators

```
public class Aoperators
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
```

```
        String x = "Thank", y = "You";
```

```
        System.out.println("a + b = "+(a + b));
```

```
        System.out.println("a - b = "+(a - b));
```

```
            System.out.println("x + y = "+x + y);
```

```
        System.out.println("a * b = "+(a * b));
```

```
        System.out.println("a / b = "+(a / b));
```

```
            System.out.println("a % b = "+(a % b));
```

```
    }
```

```
}
```

The Relational Operators

The following relational operators are supported by Java language.

Example: int A=10,B=20;

Operator

== (equal to) != (not equal to)

instance of
Operator

Description
Checks if the values are equal or no condition becomes t
Checks if the values are equal or not, i equal then condition

Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.

Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.

Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.

Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.

checks whether the object is an instance of a particular type (class or interface type)
(Object reference variable)
instanceof(class/interface type)

> (greater than)

< (less than)

>= (greater than or equal to)

<= (less than or equal to)

// Java program to illustrate relational operators

```
public class operators
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int a = 20, b = 10;
```

```
        boolean condition = true;
```

```
        //various conditional operators
```

```
        System.out.println("a == b :" + (a == b)); System.out.println("a
```

```
        < b :" + (a < b));
```

```
        System.out.println("a <= b :" + (a <= b)); System.out.println("a
```

```
        > b :" + (a > b));
```

```
        System.out.println("a >= b :" + (a >= b)); System.out.println("a
```

```
        != b :" + (a != b));
```

```
        System.out.println("condition==true :" + (condition == true));
```

```
    }
```

```
}
```

Bitwise Operators

Java supports several bitwise operators, that can be applied to the integer types, long, int, short, char, and byte.

Bitwise operator works on bits and performs bit-by-bit operation. *Example:*

```
int a = 60,b = 13;
```

binary format of a & b will be as follows –

a = 0011 1100 b = 0000 1101

Bitwise operators follow the truth table:

a	b	a&b	a b	a^b	~a
0	0	0	0	1	1
0	1	0	1	0	1
1	0	0	1	0	0
1	1	1	1	1	0

Operator	Description	Example	Output
& (bitwise and)	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B)	12 (in binary form : 0 0 0 0 1100)

(bitwise or)	Binary OR Operator copies a bit if it exists in either operand.	(A B)	61 (in binary form: 0011 1101)
^ (bitwise XOR)	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B)	49 (in binary form: 0011 0001)
~ (bitwise complement)	Binary Ones complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011	-61 (in binary form: 1100 0011)
<< (left shift)	The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000	240 (in binary form: 1111 0000)
>> (right shift)	The left operands value is moved right by the number of bits specified by the right	A >> 2 will give 15 which is 1111	15 (in binary form: 1111)

	operand.		
>>> (zero fill right shift)	The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>>2 will give 15 which is 0000 1111	15 (in binary form: 0000 1111)

// Java program to illustrate bitwise operators

```
public class operators
{
    public static void main(String[] args)
    {
        int a = 10; int b = 20;
        System.out.println("a&b = " + (a & b));
        System.out.println("a|b = " + (a | b));
        System.out.println("a^b = " + (a ^ b));
        System.out.println("~a = " + ~a);
    }
}
```

Logical Operators

The following are the logical operators supported by java.

Example:

A=true; B=false;

Operator	Description	Example	Output
&& (logical and)	If both the operands are non-zero, then the condition becomes true.	(A && B)	false
(logical or)	If any of the two operands are non zero, then the condition becomes true.	(A B)	true

! (logical not)	Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B)	true
-----------------	---	-----------	------

Assignment Operators

The following are the assignment operators supported by Java.

Operator	Description	Example
----------	-------------	---------

=	Assigns values from right side operand.	C = A + B will assign value of A + B into C
---	---	---

(Simple assignment operator)

+= (Add AND assignment operator)	It adds right operand to the left operand and assigns the result to left operand.	C += A is equivalent to C = C + A
-= (Subtract AND assignment operator)	It subtracts right operand from the left operand and assigns the result to left operand.	C -= A is equivalent to C = C - A
*= (Multiply AND assignment operator)	It multiplies right operand with the left operand and assigns the result to left operand.	C *= A is equivalent to C = C * A
/= (Divide AND assignment operator)	It divides left operand with the right operand and assigns the result to left operand.	C /= A is equivalent to C = C / A
%= (Modulus AND)	It takes modulus using two operands and assigns the	C %= A is equivalent to C = C % A

assignment operator)	result to left operand.	
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2

=	bitwise inclusive OR and assignment operator.	C = 2 is same as C = C 2
---	---	-----------------------------

```

public class AssignmentOperators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9; c = b;
        System.out.println("Value of c = " + c);
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a, b, e, f = " + a + "," + b + "," + e + "," + f);
    }
}

```

Ternary Operator

Conditional Operator (? :)

Since the conditional operator has three operands, it is referred as the **ternary operator**.

This operator consists of three operands and is used to evaluate Boolean expressions. The goal of the operator is to decide, which value should be assigned to the variable.

The operator is written as

variable x = (expression) ? value if true : value if false

Example:

```
public class example
{
    public static void main(String args[])
    {
        int a, b;
        a = 10;
        b = (a == 0) ? 20: 30;
        System.out.println( "b : " + b );
    }
}
```

Unary Operators

Unary operators use only one operand. They are used to increment, decrement or negate a value.

Operator Description
- Unary minus negating the values
+ Unary plus converting a negative value to positive
++ :Increment operator incrementing the value by 1
-- : Decrement operator decrementing the value by 1
! : Logical not operator inverting a boolean value

```
public class operators
{
    public static void main(String[] args)
```

```

{
    int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
    boolean condition = true;
    c = ++a;
    System.out.println("Value of c (++a) = " + c);
    c = b++;
    System.out.println("Value of c (b++) = " + c);
    c = --d;
    System.out.println("Value of c (--d) = " + c);
    c = --e;
    System.out.println("Value of c (--e) = " + c);
    System.out.println("Value of !condition =" + !condition);
}

```

Precedence of Java Operators

Operator precedence determines the grouping of operands in an expression. This affects how an expression is evaluated.

Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator.

`x = 10 + 5 * 2;`

`5 * 2` is evaluated first. Because operator `*` has higher precedence than `+`.

Category	Operator	Associativity
Postfix	>() [] . (dot operator)	Left to right
Unary	>++ - - ! ~	Right to left
Multiplicative	>* /	Left to right
Additive	>+ -	Left to right
Shift	>>> >>> <<	Left to right
Relational	>> >= < <=	Left to right
Equality	>== !=	Left to right
Bitwise AND	>&	Left to right
Bitwise XOR	>^	Left to right

Bitwise OR	>	Left to right
Logical AND	>&&	Left to right
Logical OR	>	Left to right
Conditional	?:	Right to left
Assignment	>= += -= *= /= %= >>= <<= &= ^= =	Right to left

1.5 CONTROL STRUCTURES:

Control statements control the flow of execution in a java program, based on data values and conditional logic used.

There are three main categories of control flow statements;

- ✓ *Branching Statements* : if, if-else and switch.
- ✓ *Looping statements* : while, do-while and for.
- ✓ *Jumping statements* : break, continue, return.

Branching Statements

The selection statements checks the condition only once for the program execution.

If Statement:

The if statement executes a block of code only if the specified expression is true.

If the value is false, then the if block is skipped and execution continues with the rest of the program.

The simple if statement has the following syntax: **if**

(<conditional expression>)

<statement action>

The following program explains the if statement.

```
public class programIF
{
    public static void main(String[] args)
    {
        int a = 10, b = 20;
        if (a > b)
```

```

        System.out.println("a > b");
    if (a < b)
        System.out.println("a < b");
    }
}

```

The If-else Statement

The if/else statement is an extension of the if statement.

If the condition in the if statement fails, the statements in the else block are executed.

The if-else statement has the following syntax:

```

if (<conditional expression>)
    <statement action>
else
    <statement action>

```

The following program explains the if-else statement.

```

public class ProgramIfElse
{
    public static void main(String[] args)
    {
        int a = 10, b = 20;
        if (a > b)
        {
            System.out.println("a > b");
        }
        else
        {
            System.out.println("a < b");
        }
    }
}

```

Switch Case Statement

The switch case statement is also called as multi-way branching statement

with several choices.

The switch statement begins with a keyword, followed by an expression that equates to a no long integral value.

After the controlling expression, there is a code block that contains zero or more labeled cases.

Each label must equate to an integer constant and each must be unique. When the switch statement executes, it compares the value of the controlling expression to the values of each case label.

The program will select the value of the case label that equals the value of the controlling expression and branch down that path to the end of the code block.

If none of the case label values match, then none of the codes within the switch statement code block will be executed.

Java includes a default label to use in cases where there are no matches. A nested switch within a case block of an outer switch is also allowed. When executing a switch statement, the flow of the program falls through to the next case.

So, after every case, you must insert a break statement.

The syntax of switch case is given as follows:

```
switch (<non-long integral expression>)  
{  
  
    case label1:  
  
        <statement1>  
  
    case label2:  
  
        <statement2>  
  
        ...  
  
    case labeln:  
  
        <statementn>  
  
    default:  
  
        <statement>
```

} // end switch

Looping Statements

Iteration statements execute a block of code for several numbers of times until the condition is true.

While Statement

The while statement is one of the looping constructs control statement that executes a block of code while a condition is true.

The loop will stop the execution if the testing expression evaluates to false. The loop condition must be a boolean expression.

The syntax of the while loop is

while (<loop condition>)
<statements>

```
public class ProgramWhile
{
    public static void main(String[] args)
    {
        int count = 1;
        System.out.println("Printing Numbers from 1 to 10");
        while (count <= 10)
        {
            System.out.println(count++);
        }
    }
}
```

Do-while Loop Statement

The do-while loop is similar to the while loop, except that the test condition is performed at the end of the loop instead of at the beginning.

The do—while loop executes atleast once without checking the condition. It begins with the keyword do, followed by the statements that making up the body of the loop.

Finally, the keyword while and the test expression completes the do-while loop. When the loop condition becomes false, the loop is terminated and execution

continues with the statement immediately following the loop.

The syntax of the do-while loop is

do

<loop body>

while (<loop condition>);

The following program explains the do--while statement.

```
public class DoWhileLoopDemo
{
    public static void main(String[] args)
    {
        int count = 1;
        System.out.println("Printing Numbers from 1 to 10");
        do
        {
            System.out.println(count++);
        } while (count <= 10);
    }
}
```

For Loop

The for loop is a looping construct which can execute a set of instructions for a specified number of times. It's a counter controlled loop.

The syntax of the loop is as follows:

for (<initialization>; <loop condition>; <increment expression>)

<loop body>

initialization statement executes once before the loop begins.

As long as the loop condition is true, the loop will continue. If condition is evaluated as false the first time, the loop will never be executed.

Increment (Update) expression that automatically executes after each repetition of the loop body.

All the sections in the for-header are optional. Any one of them can be left empty, but the two semicolons are mandatory.

The following program explains the for statement.

```
public class ProgramFor
```

```

{
    public static void main(String[] args)
    {
        System.out.println("Printing Numbers from 1 to 10");
        for (int count = 1; count <= 10; count++)
        {
            System.out.println(count);
        }
    }
}

```

Jumping Statements:

Jumping statements are used to transfer the flow of execution from one statement to another.

Continue Statement:

A continue statement stops the current iteration of a loop (while, do or for) and causes execution to resume at the top of the nearest enclosing loop. The continue statement can be used when you do not want to execute the remaining statements in the loop, but you do not want to exit the loop itself.

The syntax of the continue statement is

continue; // the unlabeled form

continue <label>; // the labeled form

It is possible to use a loop with a label and then use the label in the continue statement.

The label name is optional, and is usually only used when you wish to return to the outermost loop in a series of nested loops.

The following program explains the continue statement.

```

public class ProgramContinue
{
    public static void main(String[] args)
    {
        System.out.println("Odd Numbers");
        for (int i = 1; i <= 10; ++i)
        {

```

```

        if (i % 2 == 0)
            continue;
        System.out.println(i + "\t");
    }
}

```

Break Statement:

The break statement terminates the enclosing loop (for, while, do or switch statement).

Break statement can be used when we want to jump immediately to the statement following the enclosing control structure.

As continue statement, can also provide a loop with a label, and then use the label in break statement.

The label name is optional, and is usually only used when you wish to terminate the outermost loop in a series of nested loops.

The Syntax for break statement is as shown below;

break; // the unlabeled form

break <label>; // the labeled form

```

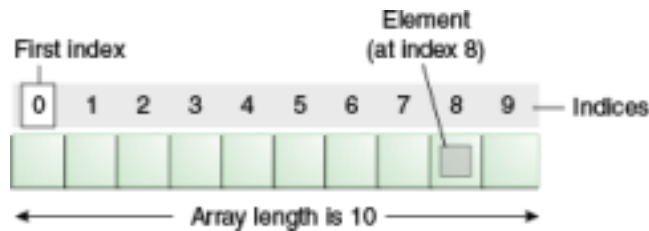
public class ProgramBreak
{
    public static void main(String[] args)
    {
        System.out.println("Numbers 1 - 10");
        for (int i = 1;; ++i)
        {
            if (i == 11)
                break;
            // Rest of loop body skipped when i is even System.out.println(i + "\t");
        }
    }
}

```

1.6 ARRAYS

Array is a collection of elements of similar data type stored in contiguous memory location.

The array size is fixed i.e we can't increase/decrease its size at runtime. It is index based and the first element is stored at 0th index.



Advantages of Array

Code Optimization: Multiple values can be stored under common name. Data retrieval or sorting is an easy process.

Random access: Data at any location can be retrieved randomly using the index.

Disadvantages of Array

Inefficient memory usage: Array is static. It is not resizable at runtime based on number of user's input.

To overcome this limitation, Java introduce collection concept.

Types of Array

There are two types of array.

One Dimensional Array

Multidimensional Array

One Dimensional Array

Declaring Array Variables

Syntax:

dataType[] arrayName; //preferred way

Or

dataType arrayName [];

Here datatype can be a primitive data type like: int, char, Double, byte etc. arrayName is an identifier.

Example: `int[] a;`

Instantiation of an Array

Array can be created using the *new* keyword.

To allocate memory for array elements we must mention the array size. The size of an array must be specified by an *int* value and not *long* or *short*. The default initial value of elements of an array is 0 for numeric types and *false* for boolean.

Syntax:

```
arrayName=new datatype[size];
```

Or

```
datatype[] arrayName=new datatype[size];
```

Example:

```
int[] a=new int[5]; //defining an integer array for 5 elements Alternatively, we  
can create and initialize array using following syntax. Syntax:
```

```
datatype[] arrayName=new datatype[] {list of values separated by comma};
```

Or

```
datatype[] arrayName={ list of values separated by comma};
```

Example:

```
int[] a={12,13,14};
```

```
int[] a=new int[] {12,13,14};
```

The built-in *length* property is used to determine length of the array i.e. number of elements present in an array.

Accessing array elements

The array elements can be accessed by using indices.

The index starts from 0 and ends at (array size-1).

Each element in an array can be accessed using *for* loop.

Example:

```
public class Main
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int a[]=new int[] {10,20,30,40}; //declaration and  
        initialization//printing array
```

```
        for(int i=0;i<a.length;i++) //length is the property of array
```

```
            System.out.println(a[i]);
```

```
}  
}
```

Sample Output:

```
10  
20  
30  
40
```

The for-each loop:

The for-each loop is used to traverse the complete array sequentially without using an index variable.

It's commonly used to iterate over an array or a Collections class.

(eg, Array- List).

Syntax:

```
for(type var:arrayName)  
{  
    Statements using var;  
}
```

Example:

```
public class Main  
{  
    public static void main(String args[])  
    {  
        int a[]=new int[]{10,20,30,40};//declaration and initialization  
        int sum=0;  
        for(int i:a) // calculate sum of array elements sum+=i;  
            System.out.println("Sum:"+sum);  
    }  
}
```

Sample Output:

```
Sum:100
```

Multidimensional Arrays

Multidimensional arrays are arrays of arrays with each element of the array

holding the reference of other array.

These are also known as Jagged Arrays.

Syntax:

```
dataType[][] arrayName=new datatype[rowsize][columnnsie];
```

```
dataType[][][] arrayName=new datatype[][][];
```

Example:

```
int[][] a=new int[3][4];
```

	Column 1	Column 2	Column 3	Column 4
Row 1	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 2	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 3	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Example:

Program to access 2D array elements

```
public class TwoDimEx
{
    public static void main(String args[])
    {
        // declaring and initializing 2D array
        int arr[][] = { {1,1,12},{2,16,1},{12,42,2} };
        // printing 2D array
        for (int i=0; i< arr.length; i++)
        {
            for (int j=0; j < arr[i].length ; j++)
            System.out.print(arr[i][j] + "\t ");
            System.out.println();
        }
    }
}
```

Sample Output:

1 1 12

2 16 1

12 42 2

Jagged Array

Jagged array is an array of arrays with different row size i.e. with different dimensions.

Example:

```
public class Main
{
    public static void main(String[] args)
    {
        int[][] a = {
                        {11, 3, 43},
                        {3, 5, 8, 1},
                        {9},
                    };
        System.out.println("Length of row 1: " + a[0].length);
        System.out.println("Length of row 2: " + a[1].length);
        System.out.println("Length of row 3: " + a[2].length);
    }
}
```

Sample Output:

Length of row 1: 3

Length of row 2: 4

Length of row 3: 1

Passing an array to a method

An array can be passed as parameter to method.

Example: Program to find minimum element in an array

```
public class Main
{
    static void min(int a[])
    {
        int min=a[0];
        for(int i=1;i<a.length;i++)
            if(min>a[i])
```

```

        min=a[i];
        System.out.println("Minimum:"+min);
    }
    public static void main(String args[])
    {
        int a[]={12,13,14,5};
        min(a); //passing array to method
    }
}

```

Sample Output:

Minimum:5

Returning an array from a method

A method may also return an array.

Example:Program to sort array elements in ascending order.

```

public class Main
{
    static int[] sortArray(int a[])
    {
        int tmp;
        for(int i=0;i<a.length-1;i++)
        { //code for sorting
            for(int j=i+1;j<=a.length-1;j++)
            {
                if(a[i]>a[j])
                {
                    tmp=a[i];
                    a[i]=a[j];
                    a[j]=tmp;
                }
            }
        }
        return(a); // returning array
    }
}

```

```

public static void main(String args[])
{
    int a[]={33,43,24,5};
    a=sortArray(a); //passing array to method
    for(int i=0;i<a.length;i++)
        System.out.println(a[i]);
    }
}

```

Sample Output:

```

5
24
33
43

```

ArrayIndexOutOfBoundsException

ArrayIndexOutOfBoundsException gets raised when an array has been accessed with an illegal index.

The index is either negative or greater than or equal to size of array.

Example:

```

public class Main
{
    public static void main (String[] args)
    {
        int[] a={1,2,3};
        System.out.println(a[3]);
    }
}

```

Run Time Error: Exception in thread "main" java.lang.

Array Index Out Of Bounds Exception: 3

ARRAYS CLASS

The Arrays class contains various static methods for sorting and searching arrays, comparing arrays, and filling array elements.

It is available in java.util.Arrays. Some of the commonly used methods in
Example:

```

import java.util.Arrays; public class
Main
{
    public static void main(String[] args)
    {
        int a[] = {43, 64, 11, 28, 32, 91, 17, 4, 2};
        System.out.println(Arrays.toString(a));
        Arrays.sort(a);
        System.out.println("Sorted Array "+Arrays.toString(a));
        int index = Arrays.binarySearch(a,32);
        System.out.println("Position of 32 in sorted arrays is => " + index);
        int[] copy = Arrays.copyOf(a, a.length); //To copy the whole array
        System.out.println("Copied array => " + Arrays.toString(copy));
    }
}

```

Sample Output:

```

[43, 64, 11, 28, 32, 91, 17, 4, 2]
Sorted Array [2, 4, 11, 17, 28, 32, 43, 64, 91]
Position of 32 in sorted arrays is => 5
Copied array => [2, 4, 11, 17, 28, 32, 43, 64, 91]

```

1.7 STRINGS

A string is a sequence of characters.

In Java, objects of the String class are immutable, which means they cannot be changed once created.

Methods in String Class:

charAt(): Returns a character at a specified position.

equals(): Compares the two given strings and returns a Boolean, that is, True or False.

concat(): Appends one string to the end of another.

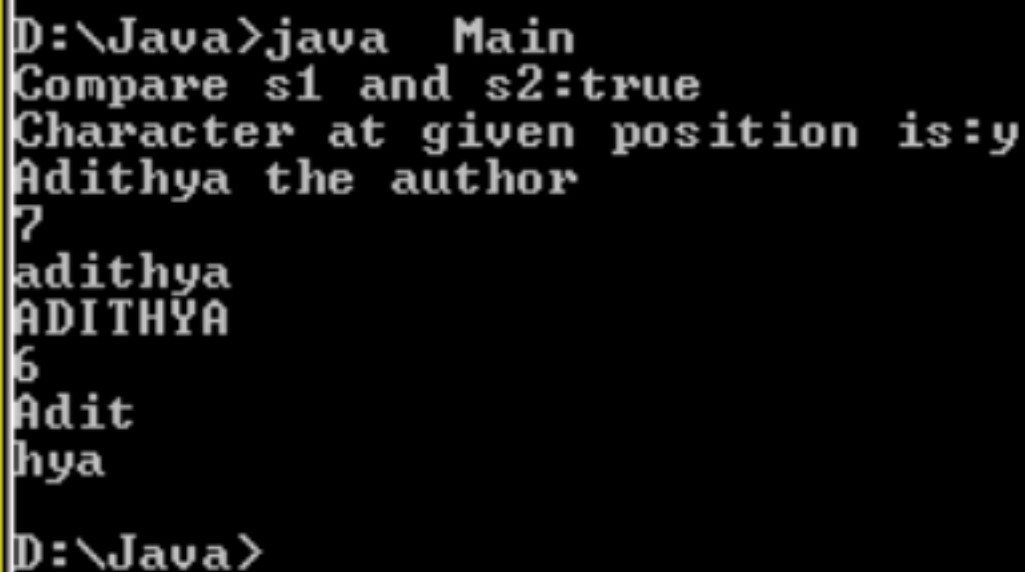
length(): Returns the length of a specified string.

toLowerCase(): Converts the string to lowercase letters.

toUpperCase(): Converts the string to uppercase letters. **indexOf():**

Returns the first found position of a character. **substring()**: Extracts the substring based on index values, passed as an argument.

```
public class Main
{
    public static void main(String []args)
    {
        String s1="Adithya";
        String s2="Adithya";
        String s3="Adi";
        boolean x=s1.equals(s2);
        System.out.println("Compare s1 and s2:"+x);
        System.out.println("Character at given position is:"+ s1.charAt (5));
        System.out.println(s1.concat(" the author"));
        System.out.println(s1.length());
        System.out.println(s1.toLowerCase());
        System.out.println(s1.toUpperCase());
        System.out.println(s1.indexOf('a'));
        System.out.println(s1.substring(0,4));
        System.out.println(s1.substring(4));
    }
}
```



```
D:\Java>java Main
Compare s1 and s2:true
Character at given position is:y
Adithya the author
7
adithya
ADITHYA
6
Adit
hya
D:\Java>
```

StringBuffer:

StringBuffer class is used to create mutable (modifiable) string.

Which means we can change the content of the StringBuffer without creating a new object every time.

Its operations are synchronized in nature & to be used in multi-threading environment. (Thread Safe)

Synchronization -> ensures that only one thread can access a critical section (like modifying data) at a time.

Methods in StringBuffer:

append(String str)	Appends the specified string to the end of the StringBuffer.
insert(int offset, String)	Inserts the specified string at the given position in the StringBuffer.
replace(int start, int end, String)	Replaces characters in a substring with the specified string.
delete(int start, int end)	Removes characters in the specified range
reverse()	Reverses the sequence of characters in the StringBuffer.

capacity() Returns the current capacity of the StringBuffer.

length()	Returns the number of characters in the StringBuffer.
charAt(int index)	Returns the character at the specified index.
setCharAt(int index, char)	Replaces the character at the specified position with a new character.
substring(int start, int end)	Returns a new String that contains characters from the specified range.
ensureCapacity(int minimum)	Ensures the capacity of the StringBuffer is at least equal to the specified minimum.
deleteCharAt(int index)	Removes the character at the specified position.
indexOf(String str)	Returns the index of the first occurrence of the specified string.

lastIndexOf(String str)	Returns the index of the last occurrence of the specified string.
toString()	Converts the StringBuilder object to a String.

```

public class StringBufferExample
{
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
        System.out.println("String: " + sb);
        sb.append(" World");
        System.out.println("After append: " + sb);
        sb.insert(6, "Java ");
        System.out.println("After insert: " + sb);
        sb.replace(6, 10, "C++");
        System.out.println("After replace: " + sb);
        sb.delete(6, 9);
        System.out.println("After delete: " + sb);
        sb.reverse();
        System.out.println("After reverse: " + sb);
        sb.reverse(); // undo reverse
        System.out.println("Undo reverse: " + sb);
        sb.setCharAt(0, 'Y');
        System.out.println("After setCharAt: " + sb);
        System.out.println("Length: " + sb.length());
        System.out.println("Capacity: " + sb.capacity());
        String result = sb.toString();
        System.out.println("Converted to String: " + result);
    }
}

```




```
cmd - Command Prompt
D:\Java>javac  StringBufferExample.java
D:\Java>java  StringBufferExample
String: Hello
After append: Hello World
After insert: Hello Java World
After replace: Hello C++ World
After delete: Hello World
After reverse: diroW olleH
Undo reverse: Hello World
After setCharAt: Yello World
Length: 12
Capacity: 21
Converted to String: Yello World
D:\Java>_
```

StringBuilder:

StringBuilder class is also used to create mutable (modifiable) string. Not thread-safe (not synchronized) ⇒ faster than StringBuffer in single-threaded environments.

Ideal for string manipulation where thread safety is not a concern.

Features	String	StringBuilder	StringBuffer
Mutability	String are immutable (creates new objects on modification)	StringBuilder are mutable(modifies in place)	StringBuffer are mutable(modifies in place)
Thread-Safe	It is thread-safe	It is not thread-safe	It is thread-safe
Performance	It is slow because it creates an object each time	It is faster (no object creation)	it is slower due to synchronization overhead
Use Case	Fixed, unchanging strings	Single-threaded string manipulation	Multi-threaded string manipulation

1.8 EXCEPTION HANDLING:

An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.

Exceptions happen when the program encounters something it cannot handle.

Common causes include:

1. Invalid input
2. File not found
3. Network errors
4. Division by zero
5. OutOfBoundArrayIndex

There are 3 types of Exceptions in Java. They are,

1. **Checked Exception:** Caught during compile time. (e.g. IOException) 2.

Unchecked Exception: Occurs at runtime. (e.g. NullPointerException) 3.

Error: Serious issues. (e.g. OutOfMemoryError)

In Java, **Exception Handling** is a mechanism to:

- ❖ Detect and manage **runtime errors** (exceptions),
- ❖ Maintain **normal flow** of the program,
- ❖ Provide **graceful error recovery** instead of crashing.

There are 5 keywords in Exception Handling. They are,

try : a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone. **catch** : a block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone.

finally : a block is used to execute the important code of the program. It is executed whether an exception is handled or not.

throw : The **throw** keyword is used to throw an exception.

throws : The **throws** keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. **Common**

Scenarios of Java Exceptions:

ArithmeticException.

```
int a=50/0;
```

NullPointerException.

```
String s=null;  
System.out.println(s.length());
```

NumberFormatException.

```
String s="abc";  
n=Integer.parseInt(s);
```

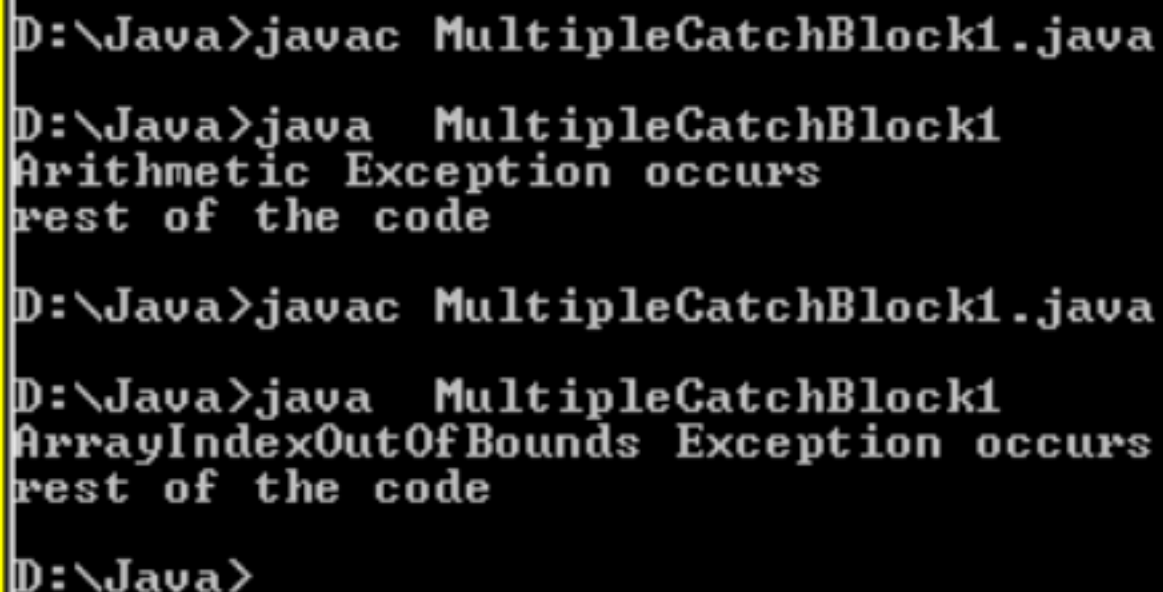
```
public class MultipleCatchBlock1  
{  
    public static void main(String[] args)  
    {  
        try  
        {
```

```

int a[]=new int[5];
    a[6]=30/0;
}
catch(ArithmeticException e)
{
System.out.println("Arithmetic Exception occurs"); }

    catch(ArrayIndexOutOfBoundsException e)
    {
System.out.println("ArrayIndexOutOfBounds Exception occurs"); }
    catch(Exception e)
    {
System.out.println("Parent Exception occurs"); }
finally()
{
System.out.println("rest of the code");
}
}
}
}

```



```

D:\Java>javac MultipleCatchBlock1.java

D:\Java>java MultipleCatchBlock1
Arithmetic Exception occurs
rest of the code

D:\Java>javac MultipleCatchBlock1.java

D:\Java>java MultipleCatchBlock1
ArrayIndexOutOfBounds Exception occurs
rest of the code

D:\Java>

```